As we will have to walk the tree up anyway, we can then retrace back our steps from the top, `R`, down to `x` and update the root pointers for all those elements: this won't influence our asymptotic performance for `findPartition`, since by retracing the same path we just double the number of steps (and constant factors are irrelevant in asymptotic analysis - see appendix B).

But as a consequence of this extra steps we take, next time we call `findPartition` on any elements in the path from `x` to `root(x)`, we know for sure that those pointers will be up-to-date and we will just need one single step to find their root.

At this point we would like to understand how many times we will need to update the root pointers, on average, for a single operation or, in amortized analysis, over a certain number `k` of operations. This is where the analysis of the algorithm gets a bit complicated.

We won't go into its details. Just know that it is proven that the running amortized time for `m` calls to `findPartition` and merge operations on a set of `n` elements will require `O(m * Ak(n))` array accesses.

Here `Ak(n)` is an approximation of the *inverse Ackermann* function, a function growing so slowly that it can be considered a constant (its value will be lower than 5 for any integer that can be stored on a computer).

So, we managed to obtain an amortized constant bound for all the operations on this data structure! If you are not impressed by this result... you should be!